



第06章 函数

张仁斌@合肥工业大学软件学院

本章主要内容



6.1 模块化代码与函数



6.2 函数定义与函数调用

6.3 函数重载

6.4 带默认形参值的函数（缺省参数）

6.5 内联函数、递归函数、系统函数

6.6 局部、全局和静态局部变量

6.7 函数抽象和逐步求精

6.8 作业与实践

模块化代码



◆ 模块化使得代码容易维护和调试（**分而治之**），使得代码可以被复用

■ 将某些相关语句的集合定义为一个**函数**，以完成某种特定计算/功能，提高代码的可重用性

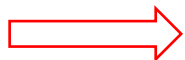
■ 例如，假设需要分别计算1~10、20~37、35~49的整数的和

```
int sum = 0;
for(int i = 1; i <= 10; i++)
    sum += i;
cout << "从1到10的累加和是" << sum << endl;
```

```
int sum = 0;
for(int i = 20; i <= 37; i++)
    sum += i;
cout << "从20到37的累加和是" << sum << endl;
```

```
int sum = 0;
for(int i = 35; i <= 49; i++)
    sum += i;
cout << "从35到49的累加和是" << sum << endl;
```

利用函数简化



```
int sum(int i1, int i2)
{
    int sum = 0;
    for(int i = i1; i <= i2; i++)
        sum += i;
    return sum;
}
```

将重复代码封装为函数体，将差异化的数据作为函数参数

```
int main() {
    cout << "从1到10的累加和是" << sum(1, 10) << endl;
    cout << "从20到37的累加和是" << sum(20, 37) << endl;
    cout << "从35到49的累加和是" << sum(35, 49) << endl;
    return 0;
}
```



◆模块化代码的优点（以sum函数为例）

函数的作用

- 调用者不需了解封装/隐藏在函数中的实现细节，使代码逻辑清晰，易读，易维护
- 如果计算累加和的函数sum有错误，这些错误均局限在sum函数中，缩小了调试范围
- 函数可以被复用，减少重复编码
- 把复杂程序模块化分为多个函数，“大事化小、分而治之”，有利于问题解决，有利于多人共同开发一个大程序，缩短程序设计周期，提高程序设计和调试的效率

```
int sum(int i1, int i2)
{
    int sum = 0;
    for(int i = 1; i <= 10; i++)
        sum += i;
    return sum;
}

int main( ) {
    cout << "从1到10的累加和是" << sum(1, 10) << endl;
    cout << "从20到37的累加和是" << sum(20, 37) << endl;
    cout << "从35到49的累加和是" << sum(35, 49) << endl;
    return 0;
}
```

本章主要内容



6.1 模块化代码与函数

6.2 函数定义与函数调用



6.3 函数重载

6.4 带默认形参值的函数（缺省参数）

6.5 内联函数、递归函数、系统函数

6.6 局部、全局和静态局部变量

6.7 函数抽象和逐步求精

6.8 作业与实践

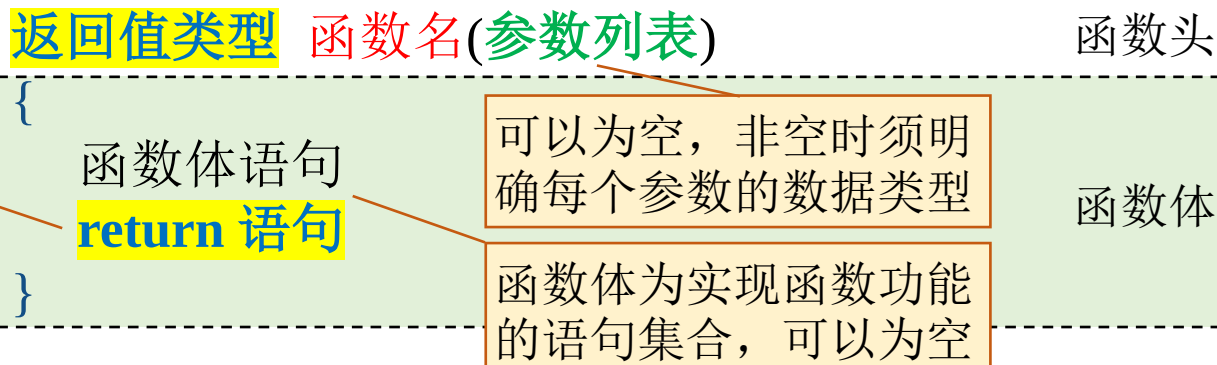


函数定义

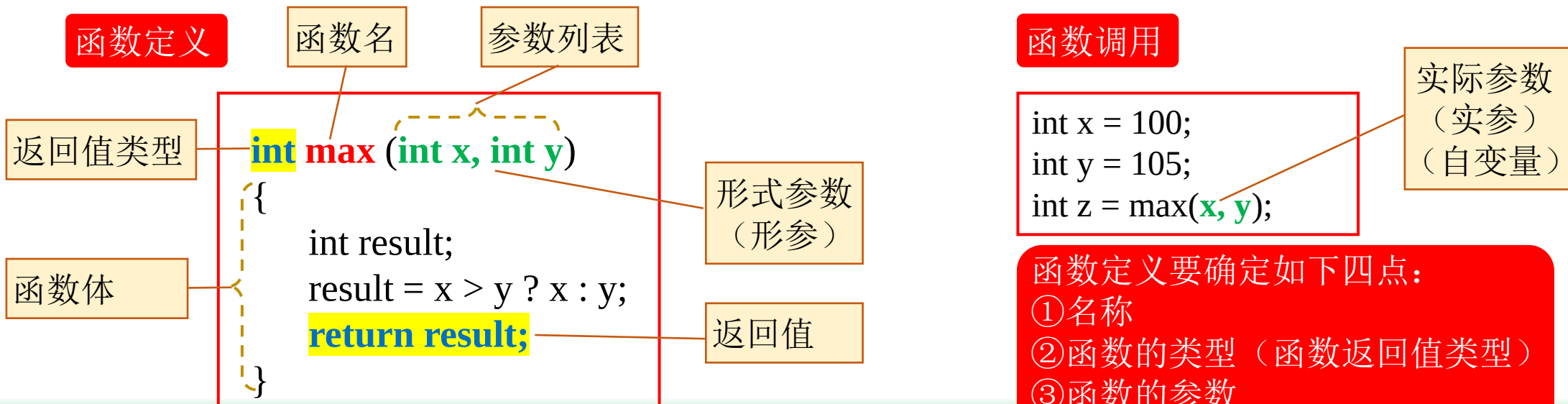
◆ 一个函数由函数名、函数参数、函数返回值类型和函数主体构成

■ 定义函数的语法：

- 【1】return语句执行后，函数即结束；
- 【2】函数体中可以有多条return语句（多个函数出口），但应尽量避免；
- 【3】return语句须与返回值类型匹配



■ 例如：取最大值函数，即输入两个整数，进行比较，将大数返回



函数定义要确定如下四点：

- ① 名称
- ② 函数的类型（函数返回值类型）
- ③ 函数的参数
- ④ 函数的功能（函数体）



无返回值函数

- ◆ 某些函数只执行指定操作，不返回一个值，则定义函数时，“返回值类型”应为“**void**”，**void函数不需要但可以使用return语句**

```
#include <iostream>
using namespace std;

void printGrade(double score)
{
    if (score >= 90.0)
        cout << 'A' << endl;
    else if (score >= 80.0)
        cout << 'B' << endl;
    else if (score >= 70.0)
        cout << 'C' << endl;
    else if (score >= 60.0)
        cout << 'D' << endl;
    else
        cout << 'F' << endl;
}
```

没有使用return语句

```
int main()
{
    cout << "Enter a score: ";
    double score;
    cin >> score;

    cout << "The grade is ";
    printGrade(score);

    return 0;
}
```

D:\Edu-CPP\chap06\ex06-01.exe

```
Enter a score: 78.5
The grade is C
```

```
#include <iostream>
using namespace std;

void printGrade(double score)
{
    if(score < 0 || score > 100)
    {
        cout << "Invalid score" << endl;
        return;
    }

    if (score >= 90.0)
        cout << 'A' << endl;

    .....
}
```

使用return语句结束本函数

关于函数中的return语句



- ◆ **return**语句终止函数的执行（函数体中后面的语句不再执行），返回到函数调用者的下一条语句
- ◆ **return**语句后面是表达式时，先计算该表达式，再**return**表达式的值
 - 例如：return 3+5;
 - 若**return**返回的值和函数返回值类型不一致，则自动将返回的值隐式转为函数返回值类型
- ◆ **void**函数中的**return**语句后面没有表达式或值，即为空
 - return;
- ◆ 函数中可以有多**return**语句

```
int max(int a, int b)
{
    if(a > b)
        return a;
    return b;
}
```




函数调用

◆ 调用一个函数就是执行函数中的代码

◆ 根据函数是否返回一个值，有两种调用函数的方式

■ 若函数返回一个值，则通常将对函数的调用当做一个值或语句来处理

```
int x = 100, y = 105;  
int z = max(x, y); // 将函数返回值赋给变量z  
cout << max(x, y); // 直接输出函数的返回值  
max(x, y);         // 当做语句，忽略函数的返回值
```

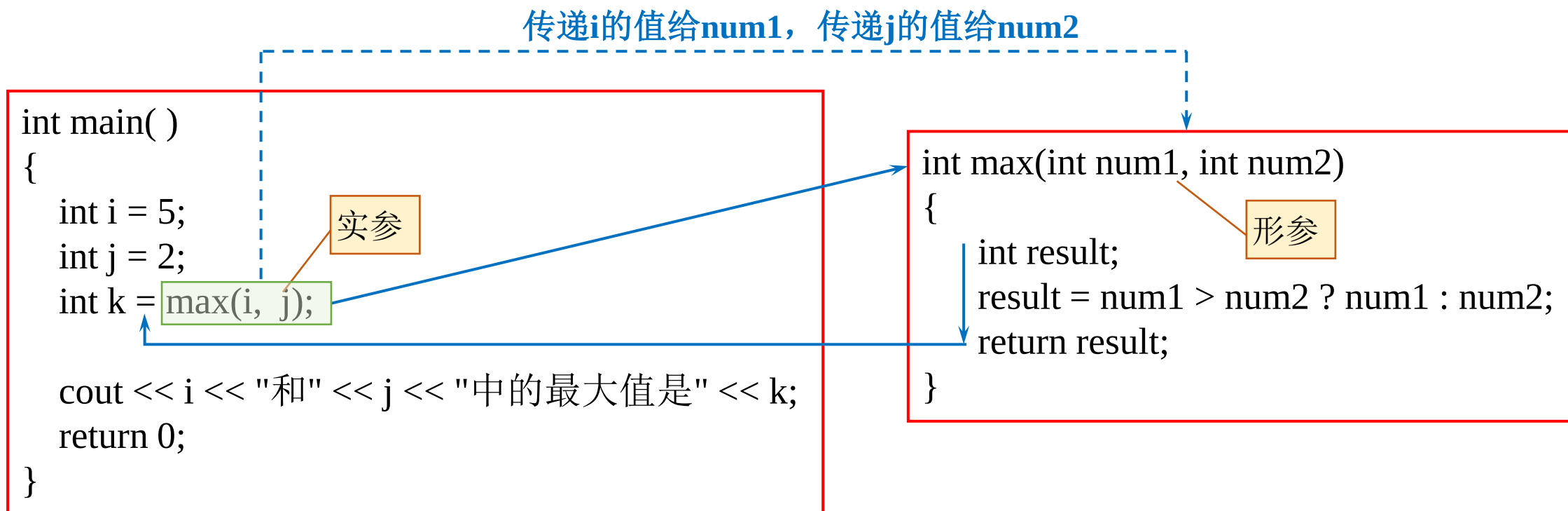
■ 若函数没有返回值（void函数），则将对函数的调用当做一个语句来处理

```
int main()  
{  
    .....  
    cout << "The grade is ";  
    printGrade(score);  
  
    return 0;  
}
```



◆ 函数调用时的控制流

- 当函数max被调用时，变量i的值5被传递给num1，变量j的值2被传递给num2，控制流转到函数max
- 当函数max中的return语句被执行时，max将控制权返回给调用者





◆ 函数调用跟踪

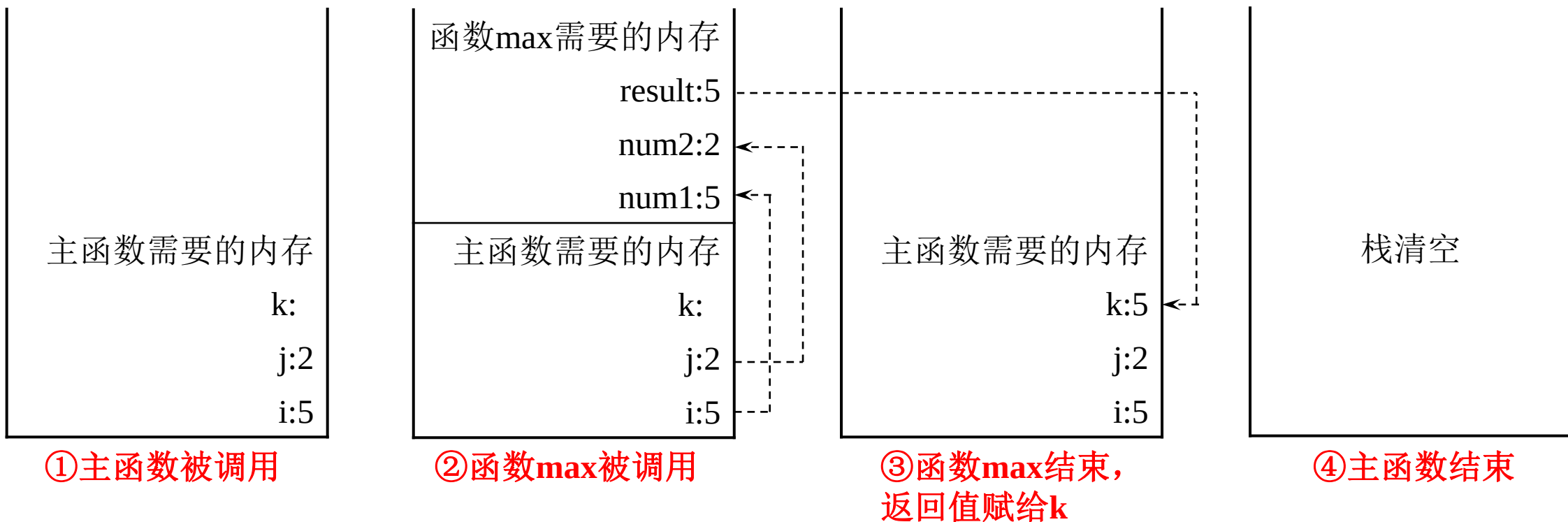
```
1  #include <iostream>
2  using namespace std;
3
4  // Return the max between two numbers
5  int max(int num1, int num2)
6  {
7      int result;
8
9      if (num1 > num2)
10         result = num1;
11     else
12         result = num2;
13
14     return result;
15 }
16
17 int main()
18 {
19     int i = 5;
20     int j = 2;
21     int k = max(i, j);
22     cout << "The maximum between " << i <<
23         " and " << j << " is " << k << endl;
24
25     return 0;
26 }
```

函数未调用时，形参并不占内存；函数调用时，形参才被分配内存；函数调用结束后，释放形参内存



◆调用栈（栈：先进后出）

函数未调用时，形参并不占内存；函数调用时，形参才被分配内存；函数调用结束后，释放形参内存



跟踪调用栈



```
int main( ) {
```

```
    int i = 5;
```

```
    int j = 2;
```

```
    int k = max(i, j);
```

```
    cout << i << "和" << j << "中的最大值是" << k;
```

```
    return 0;
```

```
}
```

定义并初始化i

```
int max(int num1, int num2) {
```

```
    int result;
```

```
    if(num1 > num2)
```

```
        result = num1;
```

```
    else
```

```
        result = num2;
```

```
    return result;
```

```
}
```

i:5

跟踪调用栈



```
int main( ) {  
    int i = 5,  
    int j = 2;  
    int k = max(i, j);  
  
    cout << i << "和" << j << "中的最大值是" << k;  
    return 0;  
}
```

定义并初始化j

```
int max(int num1, int num2) {  
    int result;  
    if(num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

j:2
i:5

跟踪调用栈



```
int main( ) {  
    int i = 5,  
    int j = 2;  
    int k = max(i, j);  
  
    cout << i << "和" << j << "中的最大值是" << k;  
    return 0;  
}
```

定义k

```
int max(int num1, int num2) {  
    int result;  
    if(num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

主函数需要的内存

k:

j:2

i:5

跟踪调用栈



```
int main( ) {  
    int i = 5,  
    int j = 2;  
    int k = max(i, j);  
  
    cout << i << "和" << j << "中的最大值是" << k;  
    return 0;  
}
```

调用函数max

```
int max(int num1, int num2) {  
    int result;  
    if(num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

主函数需要的内存

k:

j:2

i:5

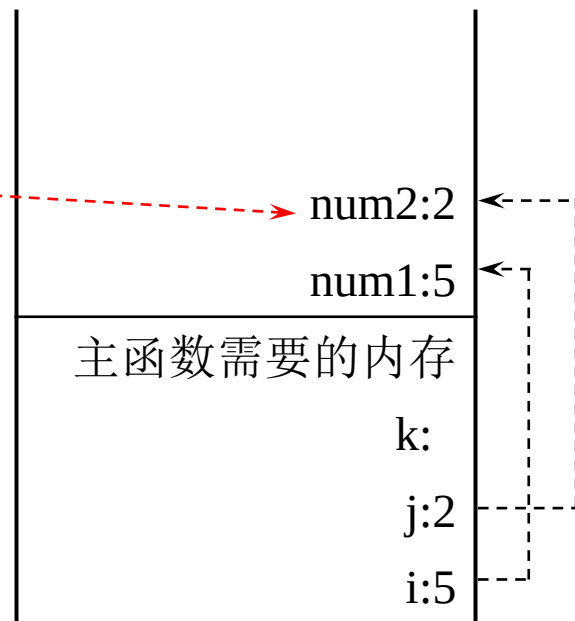
跟踪调用栈



```
int main( ) {  
    int i = 5,  
    int j = 2;  
    int k = max(i, j);  
  
    cout << i << "和" << j << "中的最大值是" << k;  
    return 0;  
}
```

传递i和j的值给num1和num2

```
int max(int num1, int num2) {  
    int result;  
    if(num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```



跟踪调用栈



```
int main( ) {  
    int i = 5,  
    int j = 2;  
    int k = max(i, j);  
  
    cout << i << "和" << j << "中的最大值是" << k;  
    return 0;  
}
```

定义result

```
int max(int num1, int num2) {  
    int result;  
    if(num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

函数max需要的内存

result:
num2:2
num1:5

主函数需要的内存

k:
j:2
i:5

跟踪调用栈



```
int main( ) {  
    int i = 5,  
    int j = 2;  
    int k = max(i, j);  
  
    cout << i << "和" << j << "中的最大值是" << k;  
    return 0;  
}
```

```
int max(int num1, int num2) {  
    int result;  
    if(num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

(num1 > num2)为真

函数max需要的内存

result:

num2:2

num1:5

主函数需要的内存

k:

j:2

i:5

跟踪调用栈



```
int main( ) {  
    int i = 5,  
    int j = 2;  
    int k = max(i, j);  
  
    cout << i << "和" << j << "中的最大值是" <  
    return 0;  
}
```

将num1的值5赋给result

```
int max(int num1, int num2) {  
    int result;  
    if(num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

函数max需要的内存

→ result:5

num2:2

num1:5

主函数需要的内存

k:

j:2

i:5

跟踪调用栈



```
int main( ) {  
    int i = 5,  
    int j = 2;  
    int k = max(i, j);  
  
    cout << i << "和" << j << "中的最大值是" << ?  
    return 0;  
}
```

返回result, 并将其值5赋给k

```
int max(int num1, int num2) {  
    int result;  
    if(num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

函数max需要的内存

result:5

num2:2

num1:5

主函数需要的内存

k:5

j:2

i:5

跟踪调用栈



```
int main( ) {  
    int i = 5,  
    int j = 2;  
    int k = max(i, j);
```

```
    cout << i << "和" << j << "中的最大值是" << k;
```

```
    return 0;
```

```
}
```

执行输出语句

```
int max(int num1, int num2) {  
    int result;  
    if(num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

主函数需要的内存

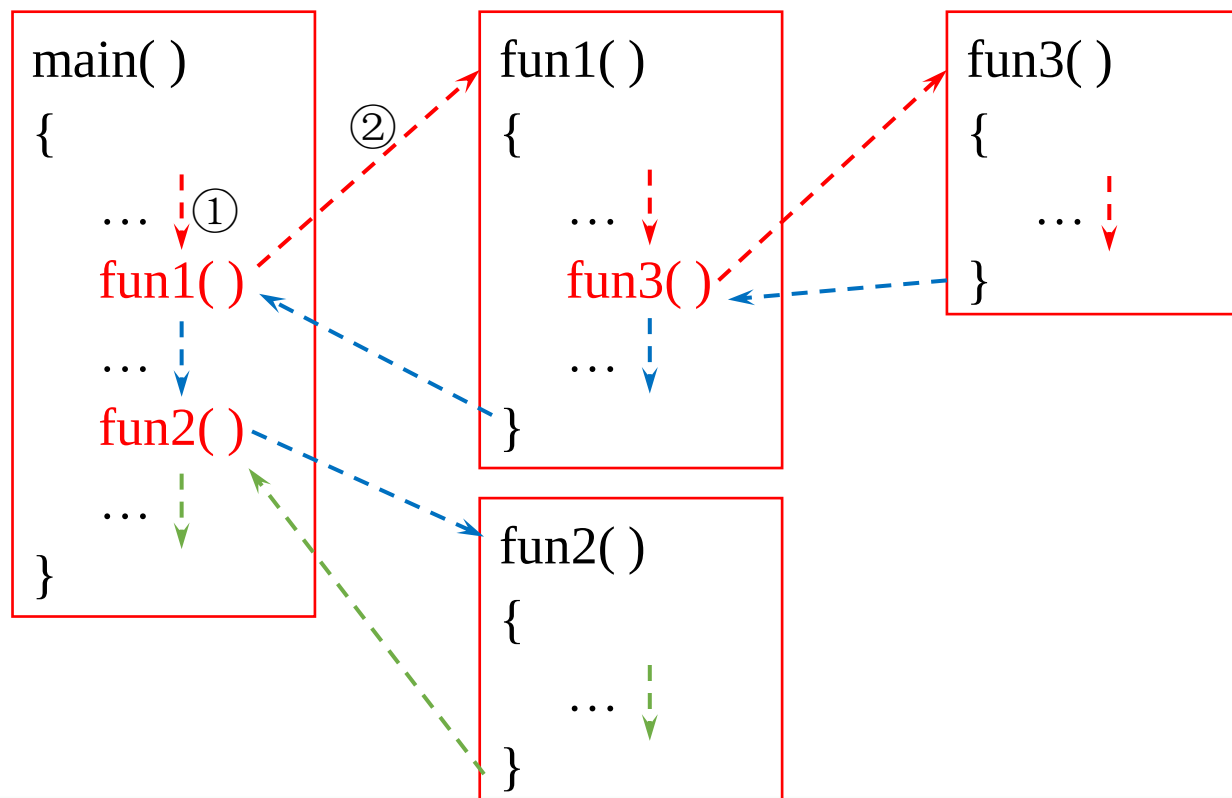
k:5

j:2

i:5



- ◆ 一个源程序文件由一个或多个函数组成
- ◆ 程序的执行从main函数开始，并回到main函数结束程序
- ◆ 函数之间可以相互调用，或调用自身（递归）
- ◆ 不能调用main函数



函数声明（函数原型）



◆ 用户自定义函数，若**定义在后，调用在前**，则必须在调用函数前对被调用函数进行**引用性声明**

■ 库函数，要在文件开头用#include将有关库函数信息“包含”进来

```
#include <iostream>
using namespace std;

int maxnum(int x, int y) ; // 函数声明（函数原型）

int main() {
    int a = 10, b = 20;
    cout << maxnum( a , b) << endl;
    return 0;
}

int maxnum(int x , int y) // 函数定义
{
    return x >= y ? x : y;
}
```

```
#include <iostream>
using namespace std;

int maxnum(int, int) ; // 可以省略变量名

int main() {
    int a = 10, b = 20;
    cout << maxnum( a , b) << endl;
    return 0;
}

int maxnum(int x , int y) // 函数定义
{
    return x >= y ? x : y;
}
```




函数参数传递

◆主调函数与被调函数数据交换：由参数传递与返回值实现

■按值传递 (pass by value)

- 以传值方式传递参数：值传递、传值
- 单向传递，实参→形参，传递的是实参的值，形参的任何变化与实参无关

■引用传递 (pass by reference)

- 以引用方式传递参数，传引用

■按址传递 (pass by pointer)

- 传地址、传址
- 与指针相关
- 双向传递，实参↔形参，传递的是实参本身



以传值方式传递参数

- ◆ 函数的能力在于其可以根据参数进行不同的工作，例如使用 **max()** 函数求任意两个整数的较大者
- ◆ 调用函数时，实参的顺序必须与形参的顺序一致（参数顺序关联）

```
#include <iostream>
using namespace std;

void nPrintLine(string msg, int n)
{
    for(int i = 0; i < n; i++)
        cout << msg << endl;
}

int main()
{
    nPrintLine("Welcome to C++!", 3);
    return 0;
}
```

识别并改正代码中的错误：

编译器自动补充 `retrun 0`; 但不是好习惯

```
#include <iostream>
using namespace std;

void nPrintLine(string msg, int n)
{
    int n = 1;
    for(int i = 0; i < n; i++)
        cout << msg << endl;
}

int main()
{
    nPrintLine(3, "Welcome to C++!");
}
```

重复定义

实参顺序错误



◆ 传值方式传递参数，传递的是实参的值，形参的改变不影响实参

```
#include <iostream>
using namespace std;

void nPrintLine(string msg, int n)
{
    for(int i = 0; i < n; i++)
        cout << msg << endl;
}

int main()
{
    nPrintLine("Welcome to C++!", 3);
    return 0;
}
```

实参为常量

```
#include <iostream>
using namespace std;

void nPrintLine(string msg, int n)
{
    for(int i = 0; i < n; i++)
        cout << msg << endl;
    n = 100;
}

int main()
{
    string msg = "Welcome to C++!";
    int n = 3;
    nPrintLine(msg, n);
    cout << "n = " << n << endl;
    return 0;
}
```

实参为变量

以引用方式传递参数



◆ 引例：实现swap(x, y)函数交换两个变量的值（传值方式传递参数）

```
#include <iostream>
using namespace std;

// Attempt to swap two variables does not work!
void swap(int n1, int n2)
{
    cout << "\tInside the swap function" << endl;
    cout << "\tBefore swapping n1 is " << n1
        << " n2 is " << n2 << endl;

    // Swap n1 with n2
    int temp = n1;
    n1 = n2;
    n2 = temp;

    cout << "\tAfter swapping n1 is " << n1
        << " n2 is " << n2 << endl;
}
```

```
int main()
{
    // Declare and initialize variables
    int num1 = 1;
    int num2 = 2;

    cout << "Before invoking the swap function, num1 is "
        << num1 << " and num2 is " << num2 << endl;

    // Invoke the swap function to attempt to swap two variables
    swap(num1, num2);

    cout << "After invoking the swap function, num1 is " << num1
        << " and num2 is " << num2 << endl;

    return 0;
}
```

D:\Edu-CPP\chap06\ex06-02.exe

```
Before invoking the swap function, num1 is 1 and num2 is 2
Inside the swap function
Before swapping n1 is 1 n2 is 2
After swapping n1 is 2 n2 is 1
After invoking the swap function, num1 is 1 and num2 is 2
```



◆ 实现swap(x, y)函数交换两个变量的值（引用方式传递参数）

```
#include <iostream>
using namespace std;
```

```
// Attempt to swap two variables does not work!
```

```
void swap(int& n1, int& n2)
```

```
{
```

```
    cout << "\tInside swap function\n";
```

```
    cout << "\tBefore swapping\n";
```

```
    cout << "n1 is " << n1 << " and n2 is " << n2 << endl;
```

```
    // Swap n1 with n2
```

```
    int temp = n1;
```

```
    n1 = n2;
```

```
    n2 = temp;
```

```
    cout << "\tAfter swapping n1 is " << n1 << " and n2 is " << n2 << endl;
```

```
    cout << "n1 is " << n1 << " and n2 is " << n2 << endl;
```

```
}
```

```
int main()
```

```
{
```

```
    // Declare and initialize variables
```

```
    int num1 = 1;
```

```
    int num2 = 2;
```

```
D:\Edu-CPP\chap06\ex06-02B.exe
```

```
Before invoking the swap function, num1 is 1 and num2 is 2
```

```
Inside the swap function
```

```
Before swapping n1 is 1 n2 is 2
```

```
After swapping n1 is 2 n2 is 1
```

```
After invoking the swap function, num1 is 2 and num2 is 1
```

```
cout << "After invoking the swap function, num1 is " << num1 << " and num2 is " << num2 << endl;
```

```
return 0;
```

```
}
```

如果形参定义为引用类型变量，那么形参是实参的别名，对形参的操作等价于对实参的操作



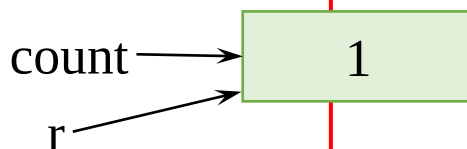
◆ 引用方式传递参数时形参是实参的**别名**（**共享同一块内存**），因此，函数内部改变形参值（更新共享的内存），也是改变实参值

◆ 引用变量的定义（给**已有**变量取别名）

➤ 一般形式：**数据类型& 引用名 = 标识符;**

```
dataType &refVar;  
dataType & refVar;  
dataType& refVar; // 更直观
```

```
int count = 1;  
int& r = count; // r和count共享相同的内存值
```



定义一个引用时，必须同时对它进行初始化，使它指向一个已存在的对象，且之后不能变更指向其它对象

使用引用
变量示例

```
#include <iostream>  
using namespace std;  
int main() {  
    int count = 1;  
    int& r = count;  
    cout << "count is " << count << endl;  
    cout << "r is " << r << endl;  
  
    r++;  
    cout << "count is " << count << endl;  
    cout << "r is " << r << endl;  
  
    count = 10;  
    cout << "count is " << count << endl;  
    cout << "r is " << r << endl;  
    return 0; }
```

D:\Edu-CPP\chap06\ex06-03.exe

```
count is 1  
r is 1  
count is 2  
r is 2  
count is 10  
r is 10
```



◆ 当使用引用方式传递参数（引用传递）时，形参和实参的数据类型必须相同，否则，不传递引用，而自动改为传值

■ 下面示例中变量x的引用被传递给函数形参n，而实参y则是将其**值传递**给n（因形参、实参数据类型不一致），**部分编译器直接报错**

```
#include <iostream>
using namespace std;
```

```
void increment(double& n) {
    n++;
}
```

```
int main() {
    double x = 1;
    int y = 1;
    increment(x);
    increment(y);
    cout << "x = " << x << endl << "y = " << y << endl;
    return 0;
}
```

```
14      increment(x);
15      increment(y);
16
17      cout << "x = " << x << endl << "y = " << y << endl;
18
19      return 0;
20  }
```

ces Compile Log Debug Find Results Close

Message	
ex06-04.cpp	In function 'int main()':
ex06-04.cpp	[Error] invalid initialization of reference of type 'double&' from expression of type 'int'
ex06-04.cpp	[Note] in passing argument 1 of 'void increment(double&)'



◆使用引用传递时，实参必须是一个变量；当使用值传递时，实参可以是变量，也可以是一个具体的值或表达式

◆给出下列代码的运行结果

函数处理的结果，可以用函数返回值返回，也可以利用参数返回

```
#include <iostream>
using namespace std;
```

```
void maxValue(int value1, int value2, int max)
{
    max = value1 > value2 ? value1 : value2;
}
```

```
int main() {
    int max = 0;
    maxValue(1, 2, max);
    cout << "max = " << max << endl;

    return 0;
}
```

D:\Edu-CPP\chap06\ex06-05.exe

max = 0

```
#include <iostream>
using namespace std;
```

```
void maxValue(int value1, int value2, int& max)
{
    max = value1 > value2 ? value1 : value2;
}
```

```
int main() {
    int max = 0;
    maxValue(1, 2, max);
    cout << "max = " << max << endl;

    return 0;
}
```

D:\Edu-CPP\chap06\ex06-05B.exe

max = 2

引用传递示例



```
#include <iostream>
#include <iomanip>
using namespace std;

void fiddle(int in1, int& in2) {
    in1 = in1 + 100;
    in2 = in2 + 100;
    cout << "[2]The values are ";
    cout << setw(5) << in1 << setw(5) << in2 << endl;
}

int main() {
    int v1 = 7, v2 = 12;
    cout << "[1]The values are ";
    cout << setw(5) << v1 << setw(5) << v2 << endl;
    fiddle(v1, v2);
    cout << "[3]The values are ";
    cout << setw(5) << v1 << setw(5) << v2 << endl;
    return 0;
}
```

> D:\Edu-CPP\chap06\ex06-06.exe

```
[1]The values are      7      12
[2]The values are    107    112
[3]The values are      7      112
```



常量引用参数

◆ 如果使用传递引用的参数，且希望参数在函数中不能改变，则应将该参数声明为常量，告诉编译器该值不能改变

■ 既利用了引用传递的高效性，又利用了值传递的安全性（不允许修改实参）

```
int max(const int& x, const int& y)
{
    return x > y ? x : y;
}
```

使用引用方式传递函数参数，不存在数据拷贝，程序执行效率更高，尤其是将复杂struct或对象作为参数时

◆ 若不需在函数中改变实参数数据，且实参数数据是基本数据类型（bool、int、long、float、double等），可以简单声明为值传递

■ 实参占用内存小，值传递开销微不足道



函数定义与调用示例1

◆ 利用 $\sin x$ 的幂级数求和公式实现函数 $\text{mySin}(x)$ ，计算精度为 10^{-8}

$$\sin x = \frac{x}{1!} - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots = \sum_{n=1}^{\infty} (-1)^{n-1} \frac{x^{2n-1}}{(2n-1)!}$$

◆ 分析

- 分子规律: $x^1, x^3, x^5, x^7, x^9, \dots$ 累乘 x^2
- 分母规律: $1!, 3!, 5!, 7!, 9!, \dots$ 累加2之后的阶乘, $n!, n!*(n+1)*(n+2)$
- 符号: + - 交错
- 循环累加至当前被累加项满足精度要求

```

#include<iostream>
#include<iomanip>
#include<cmath>
using namespace std;

const double PI = 3.14159265358;
double mySin(double x){
    double sum = 0 ;
    int n = 1;
    x = x * PI / 180;
    while(x > PI)    x = x - 2 * PI;
    while(x < -PI)  x = x + 2 * PI;
    double sqr = x * x;
    while(fabs(x) > 1e-8)
    {
        sum = (n % 4 == 1) ? sum + x : sum - x;
        x = x * sqr / ((n + 1) * (n + 2));
        n += 2;
    }

    return sum;
}

```

为什么是对4取模而不是对2取模？

人工推演一下算法是否正确，
试一下级数前3项即可

```

int main() {
    double x;
    cout << "Enter a degree:";
    cin >> x;
    cout << fixed << setprecision(6) << mySin(x);
    cout << "\n" << sin(x * PI / 180);
    return 0;
}

```

$$\sin x = \frac{x}{1!} - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots = \sum_{n=1}^{\infty} (-1)^{n-1} \frac{x^{2n-1}}{(2n-1)!}$$

D:\Edu-CPP\chap06\ex06-07.exe

```

Enter a degree:30
0.500000
0.500000

```

D:\Edu-CPP\chap06\ex06-07.exe

```

Enter a degree:780
0.866025
0.866025

```



函数定义与调用示例2

◆判断一个数是否是回文数

■回文条件：各位数字左右对称，如11，121，363，12321

◆分析

■①数据反序，构造新数据

➤利用循环遍历数中的每个数字，生成反序数

■②回文判断，新数==原数

■③输出

```
#include <iostream>
using namespace std;
```

```
int backward(unsigned n)
{
    unsigned m = 0;
    do{
        m = m * 10 + n % 10;
        n = n / 10;
    }while(n != 0);
    return m;
}
```

```
bool huiwen(unsigned n)
{
    return (n == backward(n));
}
```

```
void print(unsigned m)
{
    cout << "m = " << m << endl;
}
```

这两个函数简单到看似没必要，但可以简化主函数中的逻辑，且便于算法升级，例如，改变回文判断方法，统一调整输出格式等

```
int main()
{
    unsigned m;
    cout << "Enter a unsigned integer: ";
    cin >> m;
    if (huiwen(m))
        print(m);
    else
        cout << "不是回文数\n";

    return 0;
}
```

D:\Edu-CPP\chap06\ex06-08.exe

Enter a unsigned integer: 12321
m = 12321

D:\Edu-CPP\chap06\ex06-08.exe

Enter a unsigned integer: 256
不是回文数

本章主要内容



6.1 模块化代码与函数

6.2 函数定义与函数调用

6.3 函数重载



6.4 带默认形参值的函数（缺省参数）

6.5 内联函数、递归函数、系统函数

6.6 局部、全局和静态局部变量

6.7 函数抽象和逐步求精

6.8 作业与实践



函数重载

- ◆ C/C++ 允许在**同一源文件**中使用相同的名字命名多个函数，只需**同名函数的参数列表不同**，即函数重载
- ◆ 多个功能类似的函数用相同的函数名，方便使用和记忆
 - 功能相似
 - 实现方式形似
 - 形参类型或个数不同

```
int max(int a, int b);  
float max(float a, float b);  
double max(double a, double b);  
int max(int a, int b, int c);
```


函数重载示例



```
#include <iostream>
using namespace std;

// Return the max between two int values
int max(int num1, int num2)
{
    return num1 > num2 ? num1 : num2;
}

// Find the max between two double values
double max(double num1, double num2)
{
    return num1 > num2 ? num1 : num2;
}
```

 D:\Edu-CPP\chap06\ex06-09.exe

```
3和4中的最大数是4
1.2和3.4中的最大数是3.4
1.2和3.4及5.6中最大数是5.6
```

```
// Return the max among three double values
double max(double num1, double num2, double num3)
{
    return max(max(num1, num2), num3);
}

int main() {
    // Invoke the max function with int parameters
    cout << "3和4中的最大数是" << max(3, 4) << endl;

    // Invoke the max function with the double parameters
    cout << "1.2和3.4中的最大数是" << max(1.2, 3.4) << endl;

    // Invoke the max function with three double parameters
    cout << "1.2和3.4及5.6中最大数是" << max(1.2, 3.4, 5.6) << endl;

    return 0;
}
```

max(2, 3.4)可以这样调用max函数吗？
若可以，调用的是哪个max函数？

可以，编译器根据实参和形参的匹配确定调用哪个同名函数，编译器选择调用最匹配的max(double, double)



◆重载函数应让编译器能够识别调用的是哪个函数，否则存在编译错误：

```
#include <iostream>
using namespace std;
```

```
int maxNumber(int num1, double num2)
{
    return num1 > num2 ? num1 : num2;
}
```

```
double maxNumber(int num1, double num2)
{
    return num1 > num2 ? num1 : num2;
}
```

```
int main( ) {
    cout << maxNumber(1, 2) << endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;
```

```
void print(int i)
{
    cout << i << endl;
}
```

```
void print(int j)
{
    cout << j << endl;
}
```

```
int main( ) {
    print(1);
    return 0;
}
```

编译器不能依据不同的
返回值类型或不同的形
参名来区分重载函数

本章主要内容



6.1 模块化代码与函数

6.2 函数定义与函数调用

6.3 函数重载

6.4 带默认形参值的函数（缺省参数）



6.5 内联函数、递归函数、系统函数

6.6 局部、全局和静态局部变量

6.7 函数抽象和逐步求精

6.8 作业与实践

缺省参数



- ◆ C++ 允许在定义函数时指定参数的缺省值（默认值），函数调用时若未给出该参数对应的实参，则该参数的默认值将作为实参传递给函数

```
#include <iostream>
using namespace std;

// Display area of a circle
void printArea(double radius = 1)
{
    double area = radius * radius * 3.14159;
    cout << "area is " << area << endl;
}

int main() {
    printArea(); // 使用缺省参数1
    printArea(4);

    return 0;
}
```

D:\Edu-CPP\chap06\ex06-10.exe

```
area is 3.14159
area is 50.2654
```



◆ 函数参数列表中具有缺省值的参数应放在参数列表的末尾

```
void t1(int x, int y = 0, int z);           // 非法  
void t2(int x = 0, int y = 0, int z);       // 非法  
  
void t3(int x, int y = 0, int z = 0);       // 合法  
void t4(int x = 0, int y = 0, int z = 0);   // 合法
```

◆ 调用具有缺省参数的函数时，若某个缺省参数未给出，则其后面全部参数都不能给出

```
t3(1, , 20); // 非法  
t4(, , 20);  // 非法  
  
t3(1);        // 合法, x赋值1, 其它赋值缺省值  
t4(1, 2);     // 合法, z赋值缺省值
```



◆ 应避免重载与默认值导致的二义性

```
int max(int a, int b);  
int max(int a, int b, int c);
```

```
int max(int a, int b);  
int max(int a, int b, int c=10);  
cout << max(5, 7); ❌
```

本章主要内容



6.1 模块化代码与函数

6.2 函数定义与函数调用

6.3 函数重载

6.4 带默认形参值的函数（缺省参数）

6.5 内联函数、递归函数、系统函数



6.6 局部、全局和静态局部变量

6.7 函数抽象和逐步求精

6.8 作业与实践

内联函数



- ◆使用函数模块化实现程序，使程序易读、易维护，但函数调用存在额外的运行时间开销（将参数和CPU寄存器压入调用栈，以及在函数间切换控制所花费的时间）
- ◆使用内联函数可避免函数调用开销
 - 编译器将内联函数的代码复制到每个调用点，即内联函数的扩展（编译后，函数不存在了），从而避免函数调用
- ◆内联函数的定义使用关键字**inline**，适合将短函数定义为内联函数，避免将复杂的长函数定义为内联函数
 - 若将长函数定义为内联函数，编译器将自动忽略其**inline**关键字

```
inline void printNum(int x)
{
    cout << "The number is " << x << endl;
}
```


内联函数的定义与调用示例



```
#include <iostream>
using namespace std;
```

内联函数与普通函数
代码层面的唯一区别

```
inline void f(int month, int year)
{
    cout << "month is " << month << endl;
    cout << "year is " << year << endl;
}

int main()
{
    int month = 11, year = 2024;
    f(month, year); // Invoke inline function
    f(10, 2025);    // Invoke inline function

    return 0;
}
```

编译器进行
内联函数扩展

```
#include <iostream>
using namespace std;
```

等效编译此代码（无
自定义函数的调用）

```
int main()
{
    int month = 11, year = 2024;
    cout << "month is " << month << endl;
    cout << "year is " << year << endl;
    cout << "month is " << 10 << endl;
    cout << "year is " << 2025 << endl;

    return 0;
}
```

D:\Edu-CPP\chap06\ex06-11.exe

```
month is 11
year is 2024
month is 10
year is 2025
```

递归函数



◆ 递归基本思想

■ 把**规模大**的问题转化为**规模小**的相似的**子问题**来解决

◆ 递归的表现

■ 在函数定义中，函数直接或间接**调用自己**，称**递归调用**，这类函数称为递归函数

◆ 例如：求n!

$$n! = \begin{cases} (n-1)! * n & n > 1 \\ 1 & n = 0 \text{ 或 } n = 1 \end{cases}$$



$$f(n) = \begin{cases} f(n-1) * n & n > 1 \\ 1 & n = 0 \text{ 或 } n = 1 \end{cases}$$

递归函数

递归出口

递推

```
f(6)
=> 6 * f(5)
=> 6 * (5 * f(4))
=> 6 * (5 * (4 * f(3)))
=> 6 * (5 * (4 * (3 * f(2))))
=> 6 * (5 * (4 * (3 * (2 * f(1)))))
=> 6 * (5 * (4 * (3 * (2 * 1))))
=> 6 * (5 * (4 * (3 * 2)))
=> 6 * (5 * (4 * 6))
=> 6 * (5 * 24)
=> 6 * 120
=> 720
```

回归



```
#include<iostream>
using namespace std;
```

```
long f(int n)
{
    if(n == 0 || n == 1)
        return 1;
    return f(n - 1) * n;
}
```

```
int main()
{
    int n;
    cout << "请输入一个整数: ";
    cin >> n;
    if(n < 0)
    {
        cout << "输入数据无效! " << endl;
        return -1;
    }
    cout << f(n) << endl;
    return 0;
}
```

```
>_ D:\Edu-CPP\chap06\ex06-12.exe
```

```
请输入一个整数: 5
120
```



◆ “有去”

- 问题可以分解为若干个规模较小的子问题
- 问题与子问题，规模不同，但求解思路完全一样

◆ “有回”

- 存在递归终止条件，即存在递归出口

◆ 递归函数三部曲

- 函数返回值、函数参数
- 递归逻辑
- 终止条件

```
long f(int n)
{
    if(n == 0 || n == 1)
        return 1;
    return f(n - 1) * n;
}
```

递归函数应用示例：青蛙跳台阶问题



◆ 已知：青蛙一次可以跳上1级台阶，也可以跳上2级

◆ 求解：青蛙跳上n级的台阶总共有多少种跳法

◆ 分析



■ 第1种跳法

➢ 第1次跳1个台阶，那么剩下n-1个台阶没跳，剩下的n-1个台阶的跳法有f(n-1)种

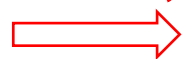
■ 第2种跳法

➢ 第1次跳2个台阶，那么剩下n-2个台阶没跳，剩下的n-2个台阶的跳法有f(n-2)种

$$f(n) = f(n-1) + f(n-2)$$

如果n=1, f(1)=1
如果n=2, f(2)=2

n ≤ 2时



f(n) = n



```
#include<iostream>
using namespace std;

long FrogJump(long n)
{
    if(n <= 2)
        return n;
    return FrogJump(n - 1) + FrogJump(n - 2);
}
```

```
int main()
{
    int n;
    cout << "请输入台阶数量: ";
    cin >> n;
    if(n <= 0)
    {
        cout << "输入数据无效! " << endl;
        return -1;
    }
    cout << "跳法数量: " << FrogJump(n) << endl;
    return 0;
}
```

 D:\Edu-CPP\chap06\ex06-13.exe

请输入台阶数量: 3
跳法数量: 3

【1】手工推演（测试）台阶数为1、2、3、4时跳法数量与算法正确性；

【2】递归函数算法简洁，但自己调用自己导致大量函数调用，产生较大的时间和空间开销

汉诺塔问题



◆有三根针A、B、C，A针上有N个大小不同的盘子，大的在下，小的在上；把这N个盘子从A针移到C针，在移动过程中可以借助B针

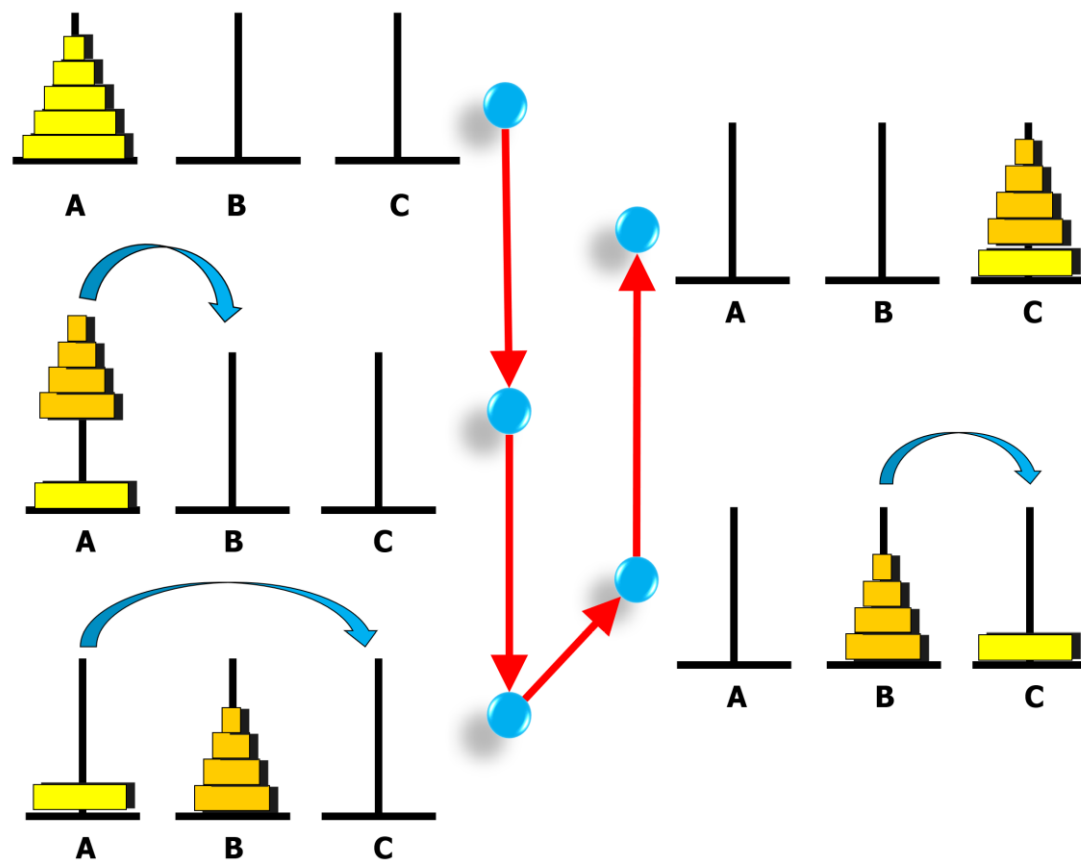
■要求：(1)每次只允许移动一个盘，(2)在移动过程中在三根针上的盘子都要保持大盘在下，小盘在上

■分析：该问题可分解为下面三个步骤：

- ①将A上n-1个盘子移到B针上（借助C针）
- ②把A针上剩下的一个盘子移到C针上
- ③将n-1个盘子从B针移到C针上（借助A针）

■上面三个步骤包含两种操作：

- ①将多个盘子从一根针移到另一根针，这是一个递归过程，用hanoi函数实现
- ②将1个盘子从一根针上移到另一针上，该过程用move函数实现



系统函数



◆ Reference - C++ Reference

■ <https://cplusplus.com/reference/>

#include<iostream>
#include <cmath>
using namespace std;

← → ↻ 🏠

🔒 <https://cplusplus.com/reference/> 📄 ☆ 🔍 搜索

⚙️ 最常访问 🌐 新手上路

Reference

▶ C library:

▶ Containers:

▶ Input/Output:

▶ Multi-threading:

▶ Other:

Reference

▼ C library:

<cassert> (assert.h)

<cctype> (ctype.h)

<cerrno> (errno.h)

<cfenv> (fenv.h) C++11

<cfloat> (float.h)

<stdinttypes> (inttypes.h) C++11

<iso646> (iso646.h)

<climits> (limits.h)

<locale> (locale.h)

<cmath> (math.h)

<setjmp> (setjmp.h)

<signal> (signal.h)

header

<cmath> (math.h)

C numerics library

Header <cmath> declares a set of functions to compute comr

fx Functions

Trigonometric functions

cos

Compute cosine (function)

sin

Compute sine (function)

本章主要内容



6.1 模块化代码与函数

6.2 函数定义与函数调用

6.3 函数重载

6.4 带默认形参值的函数（缺省参数）

6.5 内联函数、递归函数、系统函数

6.6 局部、全局和静态局部变量

6.7 函数抽象和逐步求精

6.8 作业与实践



局部变量与全局变量



◆ C/C++中，变量可以定义为局部、全局或静态局部变量。**变量的作用域**是能使用该变量的程序范围，任何变量都是**先定义后使用**

■ 函数**内部**定义的变量称为局部变量（local variable），仅在定义局部变量的函数内部可以访问该变量

➤ 一个局部变量的作用域从它的定义开始，直至包含它的**程序块**结束为止

➤ 函数**参数实质上是局部变量**，其作用域覆盖整个函数

■ 全局变量定义在**所有函数之外**（不在任何函数内），可被其作用域内的所有函数访问

➤ 全局变量的作用域是从定义变量的位置到本源文件结束（其它源文件中可以使用extern）

➤ 全局变量方便了数据的传递、共享，但增加了代码的耦合性（多个函数都依赖于该全局变量，数据异常却难以排查谁修改的），**尽量避免使用全局变量**，但使用全局常量没问题

■ 局部变量没有缺省值，**全局变量的缺省值为0**

局部变量与全局变量作用域示例



```
#include <iostream>
using namespace std;
```

```
void t1(); // 函数原型
void t2();
void t3();
```

```
int main()
{
    t1();
    t2();
    t3();

    return 0;
}
```

```
int y; // 全局变量，缺省值0
```

```
> D:\Edu-CPP\chap06\ex06-14.exe
[1]x = 1, y = 0
[2]x = 5, y = 1
[3]y = 9
```

```
void t1()
```

```
{
```

```
    int x = 1; // t1()中局部变量
```

```
    cout << "[1]x = " << x << ", y = " << y << endl;
```

```
    x++; // 更新t1()中局部变量
```

```
    y++; // 更新全局变量
```

```
}
```

影响t2()中输出的y值

```
void t2()
```

```
{
```

```
    int x = 5; // t2()中局部变量
```

```
    cout << "[2]x = " << x << ", y = " << y << endl;
```

```
}
```

可以使用作用域限定符访问全局变量：
cout << "[3]y = " << ::y << endl;

```
void t3() {
```

```
    int y = 9; // t3()中局部变量屏蔽同名全局变量
```

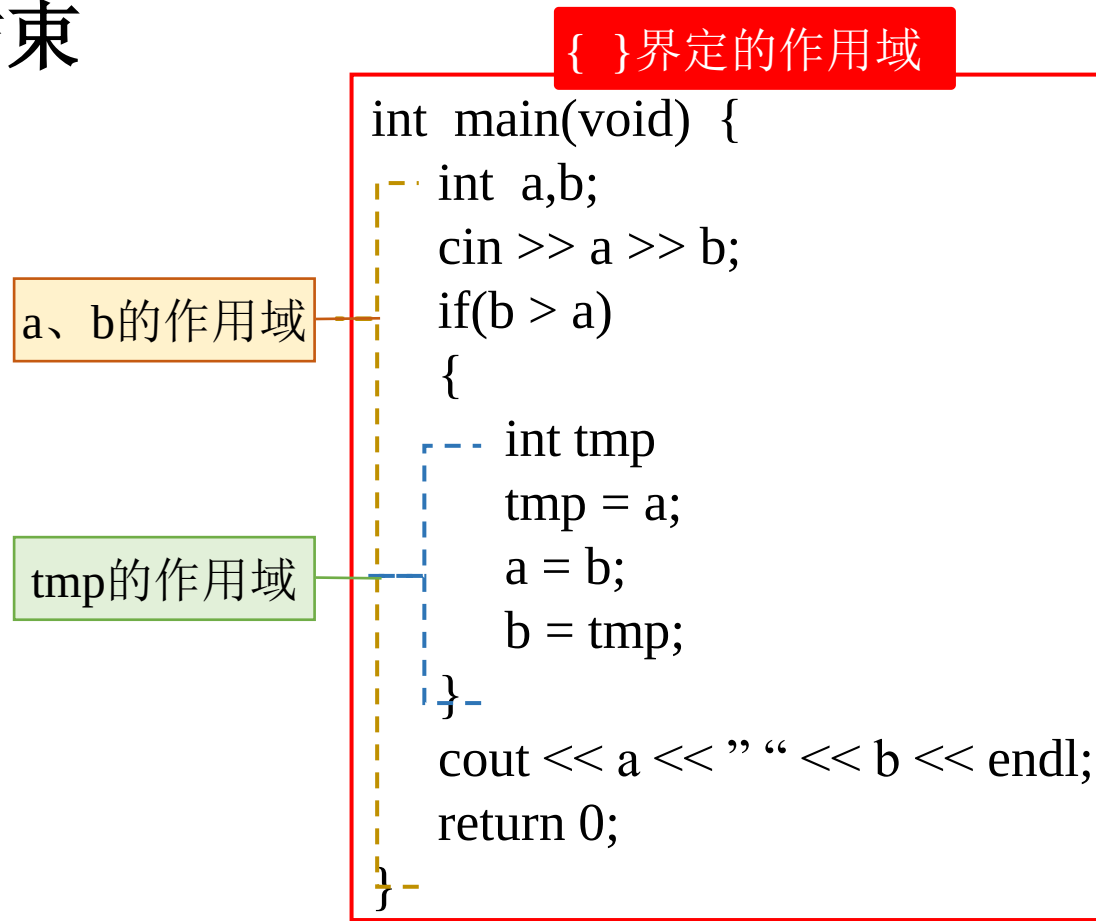
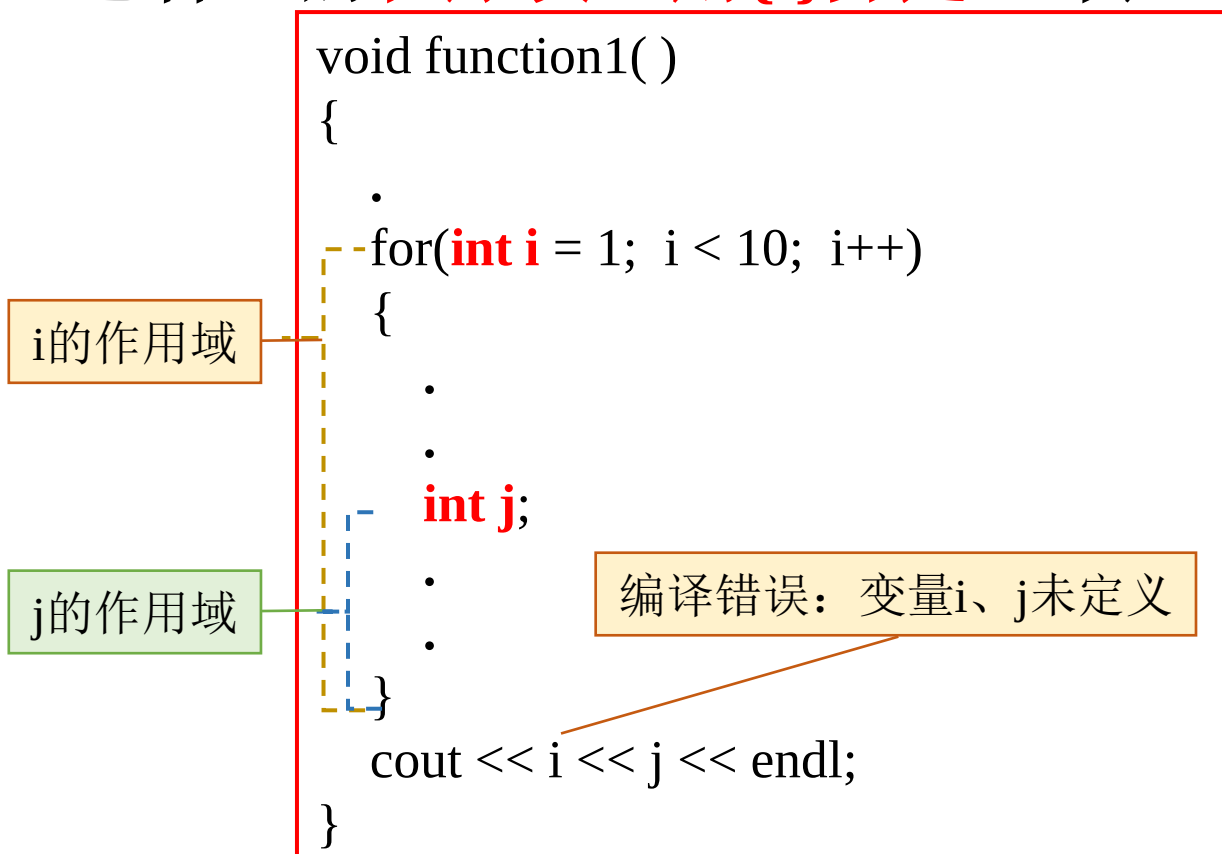
```
    cout << "[3]y = " << y << endl;
```

```
}
```

for循环中变量的作用域



- ◆ **for**循环初始化部分（表达式1）定义的变量，其作用域覆盖整个循环
- ◆ 循环体中定义的变量，其作用域在循环体内，从定义的位置开始，到包含它的**程序块（用{ }界定）**末尾结束





◆ 函数中多个非嵌套（并列）的程序块可以多次定义同名变量

```
void function1( )
{
    int x = 1;
    int y = 1;

    for(int i = 1; i < 10; i++)
    {
        x += i;
    }

    for(int i = 1; i < 10; i++)
    {
        y += i;
    }
}
```

非嵌套程序块定义同名变量i

◆ 嵌套程序块可以多次定义同名变量，是容易造成错误的坏习惯

```
void function2( )
{
    int i = 1;
    int sum = 0;

    for(int i = 1; i < 10; i++)
    {
        sum += i;
    }

    cout << i << endl; // i == 1
    cout << sum << endl;
}
```

嵌套程序块定义同名变量i

静态局部变量



- ◆ 函数执行结束（退出函数）后，函数中所有局部变量都会被销毁（释放占用的内存），这些变量也称为**自动变量**。有时候也需要保留局部变量的值，以便下次调用函数时使用，此时可以使用静态局部变量
- ◆ 在程序的整个生命周期（从开始运行到退出），静态局部变量一直驻留在内存中（**常驻内存，只初始化一次**）
- ◆ 静态局部变量的定义是在普通变量定义的前面使用关键字**static**
 - 静态局部变量的作用域与普通局部变量一样，仅限于变量被定义的函数或块内

```
static int x = 1;  
static float y;
```

静态局部变量应用示例



```
#include <iostream>
using namespace std;

void t1( ); // 函数声明（原型）

int main( )
{
    t1( );
    t1( );

    return 0;
}
```

```
void t1( )
{
    static int x = 1;
    int y = 1;
    x++;
    y++;
    cout << "x is " << x << endl;
    cout << "y is " << y << endl;
}
```

D:\Edu-CPP\chap06\ex06-15.exe

```
x is 2
y is 2
x is 3
y is 2
```

本章主要内容



6.1 模块化代码与函数

6.2 函数定义与函数调用

6.3 函数重载

6.4 带默认形参值的函数（缺省参数）

6.5 内联函数、递归函数、系统函数

6.6 局部、全局和静态局部变量

6.7 函数抽象和逐步求精

6.8 作业与实践



函数实例：十六进制转换为十进制



◆编写程序将一个十六进制数转换为一个十进制数

◆分析

■键盘输入一个十六进制数，保存为string字符串，然后遍历该字符串，将每一位都转换为十进制的数，十六进制数的第i位乘以 16^i ，然后累加所有的数，从而得到最终的十进制数

■例如，假设输入的十六进制数是AB8C，则对应的十进制数是：

3	2	1	0
A	B	8	C

$$10 \times 16^3 + 11 \times 16^2 + 8 \times 16^1 + 12 \times 16^0 = 43916$$

```

#include <iostream>
#include <string>
#include <cctype>
using namespace std;

// Converts a hex number as a string to decimal
int hex2Dec(const string& hex);

// Converts a hex character to a decimal value
int hexCharToDecimal(char ch);

int main()
{
    // Prompt the user to enter a hex number as a string
    cout << "Enter a hex number: ";
    string hex;
    cin >> hex;

    cout << "The decimal value for hex number " << hex
         << " is " << hex2Dec(hex) << endl;

    return 0;
}

```

用循环的方式巧妙实现 16^i

```

int hex2Dec(const string& hex)
{
    int decimalValue = 0;
    for (unsigned i = 0; i < hex.size(); i++)
        decimalValue = decimalValue * 16 +
            hexCharToDecimal(hex[i]);

    return decimalValue;
}

int hexCharToDecimal(char ch)
{
    ch = toupper(ch); // Change it to uppercase
    if (ch >= 'A' && ch <= 'F')
        return 10 + ch - 'A';
    else // ch is '0', '1', ..., or '9'
        return ch - '0';
}

```

$$10 \times 16^3 + 11 \times 16^2 + 8 \times 16^1 + 12 \times 16^0$$

D:\Edu-CPP\chap06\ex06-16.exe

```

Enter a hex number: ab8c
The decimal value for hex number ab8c is 43916

```

函数抽象



- ◆ 函数抽象是将函数的使用 and 实现分离，函数实现细节**封装**在函数内，对函数调用者隐藏，函数调用者无需了解函数是如何实现的，函数功能的具体实现算法发生变化时，只要函数名和函数参数列表不发生变化，就不会影响调用者对函数的使用
 - 例如库函数`pow(a, b)`、`rand()`等对用户而言，就是一个“黑盒”
- ◆ 函数抽象的思想可以用于程序开发过程
 - 当编写一个复杂大程序时，可以使用**分而治之**的策略（也称作**逐步求精**），即：将原问题分解为若干子问题，子问题可以进一步分解为更小的、易于处理的问题



程序设计：自顶向下，逐步细化

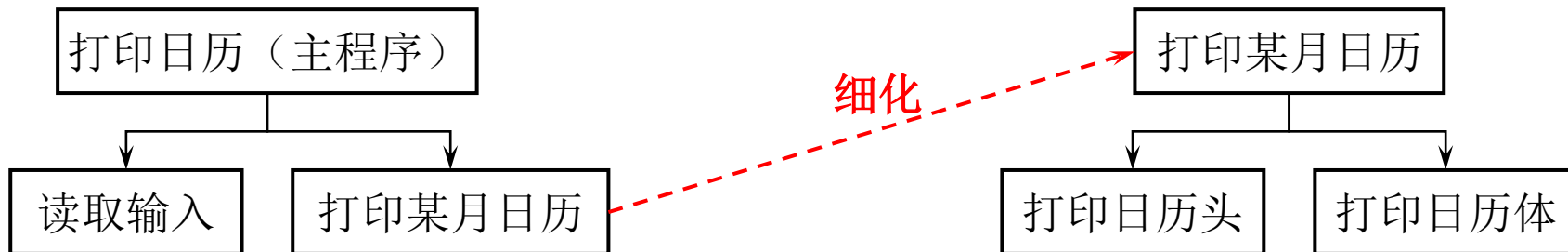
- ◆ 编写程序提示用户输入年份和月份，显示此月的完整月历

日历头 →

```
D:\Edu-CPP\chap06\ex06-17.exe
Enter full year (e.g., 2001): 2025
Enter month in number between 1 and 12: 8
August 2025
-----
Sun Mon Tue Wed Thu Fri Sat
    1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
```

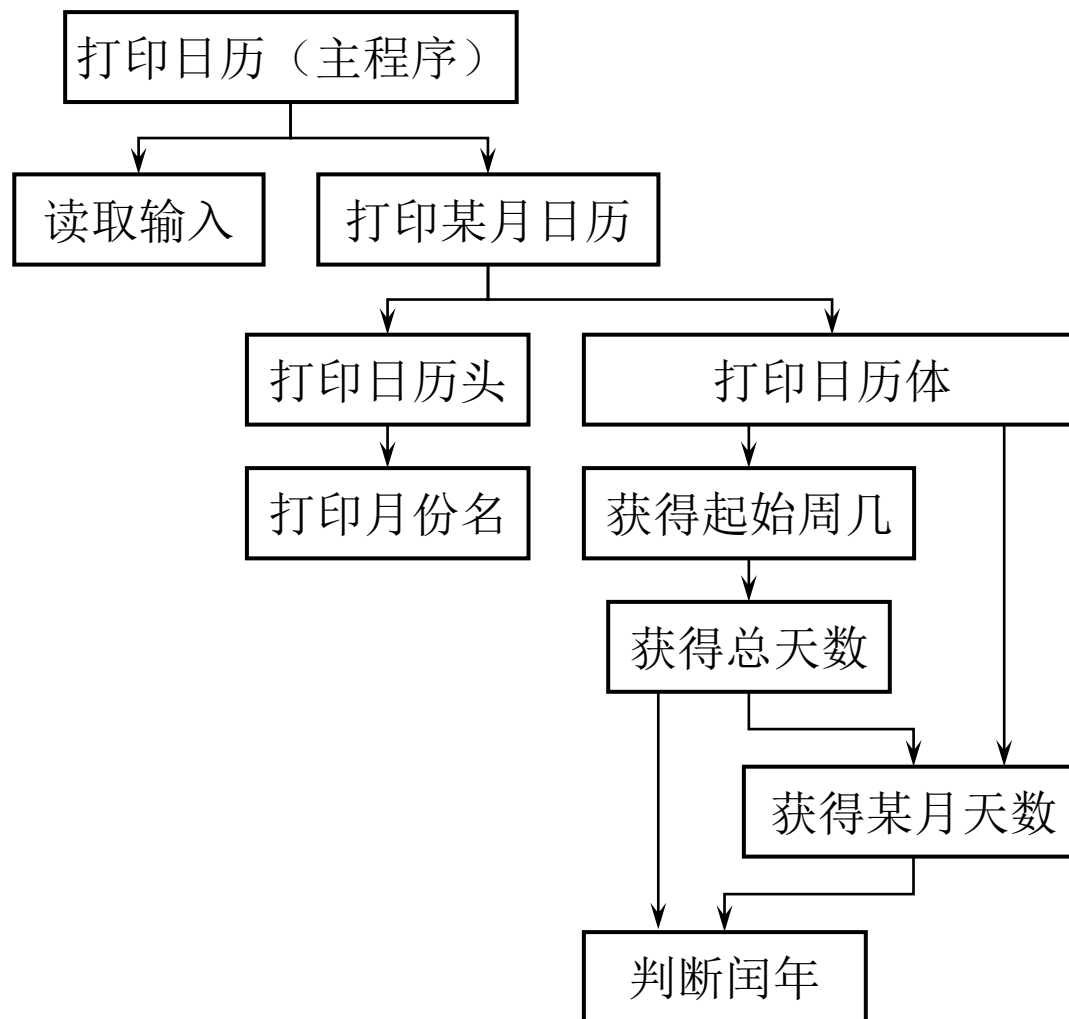
日历体 →

- ◆ 根据函数抽象思想，将抽象设计与实现细节分离，先总体设计，再细节设计（问题逐步分解、细化）





◆ 自顶向下，逐步细化



程序实现：自顶向下或自底向上



- ◆一般将每个子问题实现为一个函数，简单的子问题（例如读取输入）可以作为代码片段合并到某个函数中
- ◆自顶向下方法
 - 指按结构图，从上到下实现结构图中的每个函数，对于未实现的函数，可以用“桩（stub）”函数代替，即简单响应调用、以便测试对其调用的不完整函数版本，以支持程序的整体开发
- ◆自底向上方法
 - 指按结构图，从下向上实现结构图中的每个函数，每实现一个函数，即编写一个测试函数（也称作驱动）测试其正确性

自顶向下实现的程序代码框架



```
#include <iostream>
#include <iomanip>
using namespace std;

void printMonth(int year, int month);
void printMonthTitle(int year, int month);
void printMonthName(int month);
void printMonthBody(int year, int month);
int getStartDay(int year, int month);
int getTotalNumberOfDays(int year, int month);
int getNumberOfDaysInMonth(int year, int month);
bool isLeapYear(int year);

void printMonth(int year, int month) // “桩” 函数
{
    cout << month << " " << year << endl;
}
```

```
int main()
{
    // Prompt the user to enter year
    cout << "Enter full year (e.g., 2001): ";
    int year;
    cin >> year;

    // Prompt the user to enter month
    cout << "Enter month in number between 1 and 12: ";
    int month;
    cin >> month;

    // Print calendar for the month of the year
    printMonth(year, month);

    return 0;
}
```

完整实现

```
#include <iostream>
#include <iomanip>
using namespace std;

// Function prototypes
void printMonth(int year, int month);
void printMonthTitle(int year, int month);
void printMonthName(int month);
void printMonthBody(int year, int month);
int getStartDay(int year, int month);
int getTotalNumberOfDays(int year, int month);
int getNumberOfDaysInMonth(int year, int month);
bool isLeapYear(int year);

int main() {
    cout << "Enter full year (e.g., 2001): ";
    int year, month;
    cin >> year;
    cout << "Enter month in number between 1 and 12: ";
    cin >> month;
    printMonth(year, month);
    return 0;
}
```

```
// Print the calendar for a month in a year
void printMonth(int year, int month)
{
    // Print the headings of the calendar
    printMonthTitle(year, month);

    // Print the body of the calendar
    printMonthBody(year, month);
}

// Print the month title, e.g., May, 1999
void printMonthTitle(int year, int month)
{
    printMonthName(month);
    cout << " " << year << endl;
    cout << "-----" << endl;
    cout << " Sun Mon Tue Wed Thu Fri Sat" << endl;
}
```



```
// Get the English name for the month
void printMonthName(int month)
{
    switch (month)
    {
        case 1: cout << "January"; break;
        case 2: cout << "February"; break;
        case 3: cout << "March"; break;
        case 4: cout << "April"; break;
        case 5: cout << "May"; break;
        case 6: cout << "June"; break;
        case 7: cout << "July"; break;
        case 8: cout << "August"; break;
        case 9: cout << "September"; break;
        case 10: cout << "October"; break;
        case 11: cout << "November"; break;
        case 12: cout << "December";
    }
}
```

```
// Print month body
void printMonthBody(int year, int month)
{
    // Get start day of the week for the first date in the month
    int startDay = getStartDay(year, month);

    // Get number of days in the month
    int numberOfDaysInMonth = getNumberOfDaysInMonth(year, month);

    // Pad space before the first day of the month
    int i = 0;
    for (i = 0; i < startDay; i++)
        cout << "  ";

    for (i = 1; i <= numberOfDaysInMonth; i++)
    {
        cout << setw(4) << i;

        if ((i + startDay) % 7 == 0)
            cout << endl;
    }
}
```

```
// Get the start day of the first day in a month
int getStartDay(int year, int month) {
    // Get total number of days since 1//1//1800
    int startDay1800 = 3;
    int totalNumberOfDays = getTotalNumberOfDays(year, month);

    // Return the start day
    return (totalNumberOfDays + startDay1800) % 7;
}

// Get the number of days in a month
int getNumberOfDaysInMonth(int year, int month) {
    if (month == 1 || month == 3 || month == 5 || month == 7 ||
        month == 8 || month == 10 || month == 12)
        return 31;

    if (month == 4 || month == 6 || month == 9 || month == 11)
        return 30;

    if (month == 2) return isLeapYear(year) ? 29 : 28;

    return 0; // If month is incorrect
}
```

```
// Get the total number of days since January 1, 1800
int getTotalNumberOfDays(int year, int month)
{
    int total = 0;

    // Get the total days from 1800 to year - 1
    for (int i = 1800; i < year; i++)
        if (isLeapYear(i))
            total = total + 366;
        else
            total = total + 365;

    // Add days from Jan to the month prior to the calendar month
    for (int i = 1; i < month; i++)
        total = total + getNumberOfDaysInMonth(year, i);

    return total;
}

// Determine if it is a leap year
bool isLeapYear(int year) {
    return year % 400 == 0 || (year % 4 == 0 && year % 100 != 0);
}
```



逐步求精的优点

- ◆ 增强程序代码的可读性
- ◆ 复用函数
- ◆ 更容易开发、调试、测试
- ◆ 更好地促进团队合作
 - 基于子问题分工协作

本章主要内容



6.1 模块化代码与函数

6.2 函数定义与函数调用

6.3 函数重载

6.4 带默认形参值的函数（缺省参数）

6.5 内联函数、递归函数、系统函数

6.6 局部、全局和静态局部变量

6.7 函数抽象和逐步求精

6.8 作业与实践



第6章作业



- (1)编写函数`int sumDigits(long n)`，计算整数`n`的数字之和。例如`sumDigits(234)`应返回9。测试该函数的程序应提示用户输入一个整数
- (2)编写函数`void reverse(int number)`及其测试程序，逆序显示输入的整数，例如输入12345，显示54321
- (3)编写函数`void displaySortedNumbers(double num1, double num2, double num3)`将3个数升序显示，编写测试程序提示用户输入3个数，并调用该函数显示排序结果
- (4)编写函数`int numberOfDaysInYear(int year)`返回指定年份的天数，编写测试程序显示2015~2025年每年的天数
- (5)编写函数`void printMatrix(int n)`显示一个 $n \times n$ 的矩阵，矩阵每个元素都是随机生成的0或1。编写测试程序提示用户输入整数`n`并显示 $n \times n$ 的矩阵



(6)编写下面两个函数:

`double footToMeter(double foot);` // 将英尺转换为米

`double meterToFoot(double meter);` // 将米转换为英尺

转换公式为: $\text{meter} = 0.305 * \text{foot}$;

编写程序调用这两个函数显示右表:

Feet	Meters	Meters	Feet
1.0	0.305	20.0	65.574
2.0	0.610	25.0	81.967
...	...	...	...
9.0	2.745	60.0	196.721
10.0	3.050	65.0	213.115

(7)编写排序函数**`void sort(double& num1, double& num2, double& num3)`**将3个数整理为升序, 编写测试程序提示用户输入3个数, 调用该函数排序并输出排序结果

(8)编写函数**`int bin2Dec(const string& binaryString)`**将二进制数转换为十进制数, 编写测试程序提示用户输入二进制数字字符串, 并显示对应的十进制数。二进制转十进制示例如下:

二进制字符串10001对应十进制数为17

$$1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2 + 1 = 17$$



(9)编写函数**string prefix(const string& s1, const string& s2)**获取两个字符串的最长公共前缀，编写测试程序提示用户输入两个字符串，并显示其最长公共前缀

(10)编写函数**string swapCase(const string& s)**将字符串中的大写字母转换成小写、小写字母转换成大写，并返回新的字符串。编写测试程序提示用户输入字符串，调用该函数并显示函数的返回值，例如：

Enter a string: I'm here

The new string is: i'M HERE